TULITA: SMART E-COMMERCE PRICE COMPARISON SYSTEM

"Compare Smart. Shop Better. Save More."

MINOR PROJECT REPORT

By

[STUDENT NAME 1] (RA……)

[STUDENT NAME 2] (RA……)

Under the guidance of

[GUIDE NAME]

In partial fulfilment for the Course

21CSC206P – ADVANCED OBJECT ORIENTED & PROGRAMMING

in

Department of Computer Science & Engineering

FACULTY OF ENGINEERING AND TECHNOLOGY

SCHOOL OF COMPUTING

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

DELHI – NCR CAMPUS

NOVEMBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this minor project report for the course 21CSC206P – ADVANCED OBJECT ORIENTED & PROGRAMMING entitled “Tulita: Smart E-Commerce Price Comparison System” is the bonafide work of [STUDENT NAME 1] (RA……) and [STUDENT NAME 2] (RA……) who carried out the work under my supervision.

This report has not been submitted for any other course or degree. The data, source code, screenshots and test results presented in this report were produced by the students and are reproducible from the project archive submitted with it.

SIGNATURE

[GUIDE NAME]
Assistant Professor
Computer Science & Engineering

SIGNATURE

[HEAD OF DEPARTMENT]
Head of the Department
Computer Science & Engineering

ABSTRACT

Shoppers routinely compare the same product across several online retailers, but the comparison is manual, error-prone and often misleading: the same model is listed under different titles, and a cheaper amount may refer to a different pack size, colour or generation of the product. Tulita is a JavaFX desktop application that performs a same-product, same-variant price comparison over a curated catalogue of genuine historical retailer observations. The catalogue contains 79 product variants, 161 priced observations and 237 cached product photographs, drawn from the publicly available ProMap product-mapping research dataset published by Macková and Pilát. Every card in the application represents one physically identical variant and carries at least two priced retailer observations, so a comparison can never be produced for an unmatched product.

The system follows a Model–View–Controller design in Java 17/21 with a clear layering: JavaFX views, a controller that keeps all slow work off the user-interface thread, service classes for search, validation and visual retrieval, and interchangeable data-access objects for local JSON files and a normalised MySQL 8 schema accessed through JDBC. Evidence integrity is enforced at load time: source row hashes, cached image hashes, retailer identity and known model numbers are re-verified before any amount is displayed, and an altered file is rejected instead of silently accepted.

In addition to text search, the application supports photo-based retrieval. A camera frame or an uploaded image is embedded by a quantised CLIP vision model executed locally with ONNX Runtime, compared by cosine similarity against the cached reference photographs, and the five nearest catalogue products are offered as suggestions. The user must confirm the exact variant before a comparison is shown; visual similarity is never treated as proof of identity and never introduces a price. No cloud service, paid API or Python runtime is involved at any stage.

A second capability turns the same catalogue into a planning tool. The Intelligent Budget Optimizer answers a different question from comparison: given a budget, a set of required or optional product categories and a set of preferences, which combination of products should be bought? It is implemented as a new layered module inside the existing application — model, data access, service, engine, controller and view — that reuses the curated product and offer records, the same price observations and the same evidence

validation as the comparison screen, and adds no product model, no database table and no dependency. The search is a Pareto-pruned dynamic programme over integer money that returns a weighted, explained recommendation in four modes (Best Value, Lowest Cost, Best Specifications and Balanced), never exceeds the budget, never silently relaxes a mandatory rule, and states in the interface when it had to fall back to a bounded search. Budgets can be entered in Indian rupees with a rate supplied by the user, because the catalogue records amounts in United States dollars and no exchange rate may be invented.

Results are reported honestly. Measured over the 79 curated products, the gap between the lowest and the highest recorded amount is USD 1,290.06 in total, with a median gap of 17.6 % of the higher amount; 53 products show a gap above 10 % and 12 products are exact ties. The complete automated suite of 133 tests passes, including database, JavaFX and ONNX inference tests. The report also states what the dataset cannot support: ratings, review counts and capture dates are absent from the source files and are therefore displayed as “Not in source” rather than estimated, and the recorded amounts are historical rather than live prices.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my guide, [GUIDE NAME], for her valuable guidance, consistent encouragement, personal caring, timely help and for providing an excellent atmosphere for doing this project. All through the work, in spite of her busy schedule, she extended cheerful and cordial support for completing this project work.

I am thankful to [HEAD OF DEPARTMENT], Head of the Department of Computer Science & Engineering, and to the faculty of the School of Computing for the laboratory facilities and the academic structure that made this work possible.

I also acknowledge Kateřina Macková and Martin Pilát, whose openly published ProMap product-mapping dataset provided the historical retailer records and product photographs used here, and the authors of the CLIP model and the ONNX Runtime, whose freely available work made fully offline image search possible inside Java.

[STUDENT NAME 1] (RA……)

[STUDENT NAME 2] (RA……)

TABLE OF CONTENTS

CHAPTER NO	CONTENTS	PAGE NO
1	**INTRODUCTION**	__
	1.1 Motivation	__
	1.2 Objective	__
	1.3 Problem Statement	__
	1.4 Challenges	__
2	**EXISTING PROBLEMS**	__
3	**REQUIREMENT ANALYSIS**	__
	3.1 Requirement Analysis	__
	3.2 Hardware Requirement	__
	3.3 Software Requirement	__
4	**ARCHITECTURE & DESIGN**	__
	4.1 System Architecture	__
	4.2 Database Design	__
	4.3 Class Design and Object-Oriented Choices	__
	4.4 Design of the Offline Image-Retrieval Path	__
5	**IMPLEMENTATION**	__
	5.1 Package and Class Layout	__
	5.2 Data Acquisition and Curation	__
	5.3 Local-File Implementation	__
	5.4 MySQL Implementation	__
	5.5 Visual Search and Camera Implementation	__
	5.6 User-Interface Implementation	__
6	**EXPERIMENT RESULTS & ANALYSIS**	__
	6.1 Test Environment	__
	6.2 Catalogue and Price Analysis	__
	6.3 Comparison Results	__
	6.4 Image-Search Results	__
	6.5 Automated Test Results	__
	6.6 Discussion and Limitations	__
7	**INTELLIGENT BUDGET OPTIMIZER**	__
	7.1 Requirement Analysis for the Planner	__
	7.2 Module Architecture	__
	7.3 The Optimisation Algorithm	__
	7.4 The Scoring Model	__

	7.5 Money, Charges and the Rupee Budget	__
	7.6 The Planner Interface	__
	7.7 What-If Analysis	__
	7.8 Impossible Requests and Failures	__
	7.9 Results	__
	7.10 Automated Tests for the Planner	__
	7.11 Limitations of the Planner	__
8	**CONCLUSION**	**__**
9	**REFERENCES**	**__**

1. INTRODUCTION

1.1 Motivation

An online shopper who wants the best available amount for a specific product usually opens three or four retail websites, searches the same model number in each of them and then compares what is found. This is slow, but the real difficulty is not speed. The same physical product is described differently by every retailer: the title order changes, the pack size is buried in a subtitle, and the model number appears only in a specification table. A shopper comparing two titles therefore frequently compares two different things — a 42-count pack against a 24-count pack, a black variant against a rose-gold one, or a 1.1 cu ft microwave against a 0.7 cu ft one — and concludes that one retailer is cheaper when, in fact, a different product is being priced.

This mismatch is the central problem that the project addresses. A price-comparison programme is only useful if it compares the same variant; otherwise the comparison is an artefact of sloppy matching. Tulita was therefore built around a curated, evidence-backed product identity, and every later feature — text search, filters, charts, image search and database storage — operates on that identity rather than on loosely similar titles. The project is also an exercise in the object-oriented ideas of the course: encapsulation of evidence, polymorphism between two storage back-ends, abstraction through interfaces, record-based immutability, exception handling that never fakes success, and concurrency that keeps a graphical interface responsive.

1.2 Objective

The objectives of the project are:

- To build a JavaFX desktop application, and not a website, that compares the recorded amounts of one exact product variant across several retailers.
- To use only genuine historical retailer records, with no simulated, generated or relabelled prices anywhere in the comparison path.
- To guarantee, by construction and by validation, that every comparison shown to the user involves at least two priced retailers for one identical variant.

- To support unrestricted text search over product name, brand, model, variant and retailer title, with filters and sorting over the whole catalogue.
- To add photo-based retrieval using a pretrained model executed locally in Java, with the user confirming the exact variant before any comparison.
- To provide two interchangeable storage modes — local JSON files and MySQL 8 over JDBC — behind one data-access interface.
- To display honestly what the source data cannot support, in particular ratings and price-over-time trends, and to state those gaps in the interface itself.
- To deliver a tested, packageable project that runs on a normal laptop without a cloud account, paid API, Python backend or internet connection after setup.

1.3 Problem Statement

Given a set of historical records in which the same product variant has been listed by two or more different retailers, design and implement a desktop application that identifies such variant groups correctly, displays each group as one comparable product, reports the lowest and highest recorded amount together with the difference between them, and proves the integrity of every figure it shows. The application must additionally allow a user to find that product by typing a name or by supplying a photograph, and must never present an unmatched listing, a guessed variant, an estimated rating or a fabricated time series as if it were real data.

1.4 Challenges

The following challenges shaped the design and are discussed again where they are solved:

- **Product identity without an official key.** Retailers do not share a product identifier. Identity had to be established from model numbers, size, capacity, pack quantity, colour and bundle contents, and conflicting evidence had to be rejected rather than reconciled by guesswork.
- **Labelled data that is partly wrong.** The research dataset provides positive and negative pair labels, but a positive label does not mean the two listings are the same variant. Every candidate used by the application was checked against both listings, and the rejects were recorded with a reason.

- **Missing fields in the source.** The source files contain no rating, no review count, no capture date and no currency column. Ratings and time-series charts therefore had to be replaced by an explicit absence statement and a chart of what genuinely exists.
- **Offline image retrieval inside Java.** A pretrained vision model has to be loaded and executed through ONNX Runtime, preprocessing has to match the model exactly, and the model file itself must be downloaded once and hash-verified so that no silent model substitution can occur.
- **A responsive desktop interface.** Loading, validation, indexing and inference may take seconds, so none of them may run on the JavaFX application thread; results must be published back safely and partially built state must never appear.
- **Honest failure.** The system is required to report a database, camera or model failure to the user instead of quietly falling back to sample data or a fake frame.

2. EXISTING PROBLEMS

Several existing ways of comparing prices were examined before designing Tulita. Each solves part of the problem and fails on another part, and the failures are the requirements that Tulita inherits.

2.1 Manual comparison across retailer websites

The most common method is to keep several browser tabs open and read the amounts by eye. It is accurate for a single product that the user already understands, but it does not scale to a shopping list, it leaves no record of what was compared, and it depends entirely on the user noticing that the two listings differ in pack size or colour. It also cannot answer the question that matters most to a student project: was the comparison performed between two identical variants at all?

2.2 Live price-comparison websites and browser extensions

Commercial comparison sites solve the manual effort by scraping or licensing retailer feeds in real time. They are the right tool for live pricing, but they are unsuitable here for three concrete reasons: they depend on continuous network access and on third-party terms of service, they require API keys or paid data plans, and their matching is a commercial black box that the project cannot verify. A college demonstration also cannot depend on a live connection or on an amount that may change between the demonstration and the viva.

2.3 Retailer search with a shared model number

Searching a manufacturer model number on each retailer site is the most reliable manual technique, and Tulita reproduces it inside the application. Its limitation is coverage: many listings omit the model number from the title, so a legitimate second retailer offer is missed. Tulita accepts such a record only when the remaining specifications — size, game count, dart count, battery type, pack size — agree, and states that basis in the review note for that product.

2.4 Image-based shopping search

Reverse-image shopping is normally offered as a cloud service: an image is uploaded, a remote model returns candidate products. This conflicts with the project constraints of no

cloud dependency, no paid API and no Python backend, and it also uploads the user's photograph. Tulita keeps the same idea but performs embedding and ranking locally with a quantised CLIP vision model executed by ONNX Runtime inside the Java process, so no image ever leaves the machine.

2.5 What the existing approaches together leave unsolved

Existing approach	Strength	Gap that Tulita addresses
Manual multi-tab browsing	Always available, no setup	No record, no variant guarantee, not repeatable
Live comparison websites	Real-time amounts	Network/API dependence, opaque matching, unverifiable
Retailer model-number search	Reliable when the model is listed	Misses listings without a model number
Cloud reverse-image search	Recognises arbitrary products	Uploads the photo, needs a paid service, not offline
Spreadsheet of copied prices	Easy to start	No validation, no hashes, silently accepts wrong variants

Table 2.1: Comparison of existing approaches and the gap addressed

2.6 Implications carried into the requirements

Taken together, the five approaches point at three requirements that shape the whole design. First, identity must be established before any amount is compared, and the evidence for that identity must be stored with the offer rather than left in a spreadsheet. Second, the comparison must be reproducible without a network connection, so the records and the photographs have to be bundled and validated locally. Third, every capability that cannot be supported by the data — a rating, a live amount or a trend over time — must be visibly absent rather than plausibly invented, because a plausible invention is exactly what makes a comparison misleading. Chapter 3 turns these into explicit functional and non-functional requirements.

3. REQUIREMENT ANALYSIS

3.1 Requirement Analysis

From the problem statement, the following functional requirements were derived:

1. The system shall display one card per curated product variant, listing the recorded amount of every retailer that listed that variant, and shall highlight the lowest recorded amount.
2. The system shall never display a product card that has fewer than two priced retailer observations.
3. The system shall support unrestricted search over name, brand, model, variant and retailer title, with category filter, store filter, “three or more stores” filter, session watchlist filter and three sort orders.
4. The system shall show, for a selected product, the lowest amount, the highest amount, the absolute difference, the percentage difference from the higher amount and the evidence for each observation.
5. The system shall re-verify, at load time, the hash of every source row and of every cached photograph, and shall reject altered evidence with a visible error.
6. The system shall allow retrieval of a product by photograph, using a locally executed pretrained model, and shall require explicit user confirmation of the variant before showing a comparison.
7. The system shall support two storage modes, local files and MySQL 8, behind a single data-access interface, without any silent fallback between them.
8. The system shall state, in the interface, when a requested attribute such as a rating is not present in the source data.
9. The system shall record a session watchlist and shall restore an ordinary browsing state when a photo filter is cleared.
10. The system shall package into a self-contained application image that needs no separate JDK installation on the target laptop.

Non-functional requirements were equally explicit: the interface must remain responsive while background work proceeds; a failure of the database, camera or model must be reported rather than worked around; credentials must never be hardcoded; every

displayed amount must be traceable to a source row; and the application must run on an ordinary laptop with about 1 GB of free memory for the Java process.

3.2 Hardware Requirement

The project was developed and verified on ordinary laptop-class hardware:

Component	Minimum used	Notes
Processor	x86-64 dual-core, 2 GHz	ONNX inference is limited to two intra-op threads by the code
Memory	4 GB system; 1 GB heap for the application	Packaged launcher sets –Xmx1024m; test forks use –Xmx340m
Disk	About 900 MB	Maven repository and target folder, plus 25 MB catalogue and 89 MB model
Display	1280 × 800 or larger	Interface designed for 1600 × 1200 and verified at 1320 × 1000
Camera (optional)	Any UVC webcam or laptop camera	Uploading a photograph is an equivalent input path
Network (optional after setup)	One-time 89 MB model download	All cataloguing, comparison and inference then work offline

Table 3.1: Hardware requirements

3.3 Software Requirement

Item	Version used	Purpose
JDK	OpenJDK 21.0.12 (source level 17)	Language and runtime
JavaFX	21.0.6 controls and swing modules	Desktop user interface
Maven	3.9.9	Build, dependency management, tests
MySQL	8.0.40 with Connector/J 8.4.0	Normalised database mode
ONNX Runtime	1.20.0 (CPU)	Local inference of the vision model
CLIP vision model	clip-vision-int8.onnx, 89,117,001 bytes	Image embeddings, SHA-256 verified
Jackson	2.18.3	Reading the curated JSON catalogue
Apache Commons CSV	1.12.0	Reading the original source CSVs
Webcam Capture	0.3.12	Camera frames on Windows and Linux
JUnit Jupiter	5.11.4	Automated tests
jpackage	Bundled with JDK 21	Self-contained application image
Editor	VS Code with Extension Pack for Java	Development environment

Table 3.2: Software requirements

No Python interpreter, web framework, cloud account, paid API key or GPU is required by any part of the delivered system.

4. ARCHITECTURE AND DESIGN

4.1 System Architecture

The application is organised as a layered Model–View–Controller system. Dependencies point downward only: a view never opens a database connection, and a data-access class never formats text for the screen. Figure 4.1 shows the layers and the two external artefacts they depend on.

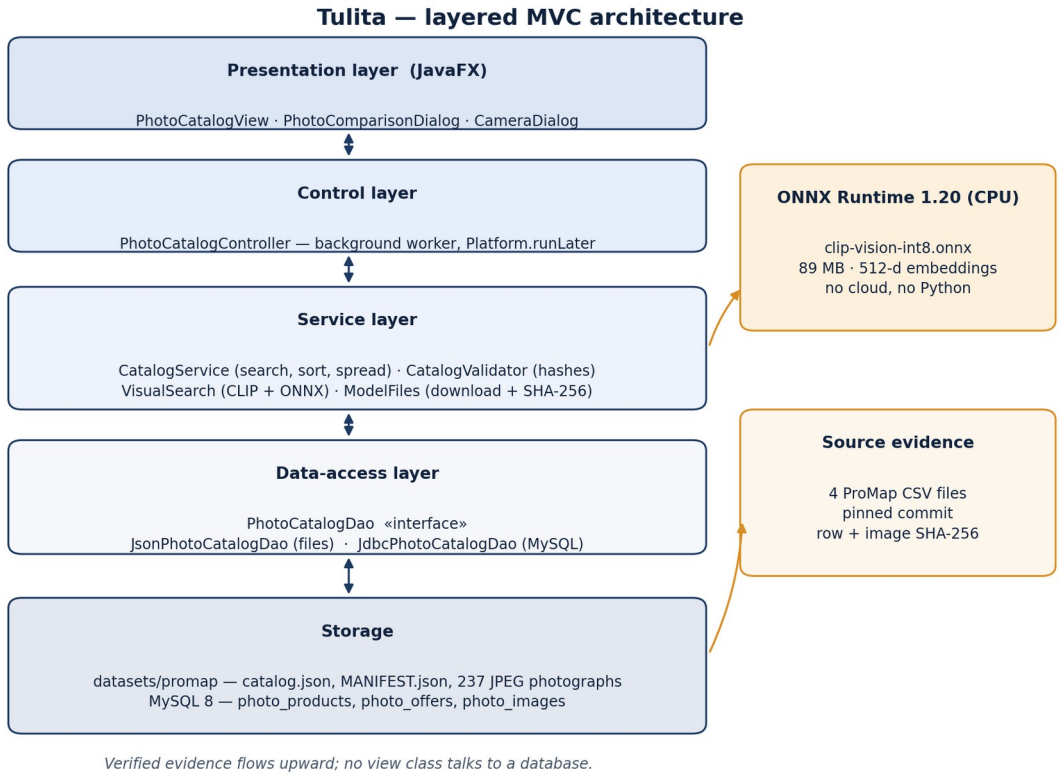


Figure 4.1: Layered architecture of Tulita, with the local inference runtime and the original source evidence shown at the side

The presentation layer contains the browsing view, the comparison sheet and the camera dialog. The control layer owns a single background executor thread, submits every slow operation to it, and publishes results back with Platform.runLater so that no JavaFX node is ever mutated from a worker thread. The service layer holds the searchable catalogue, the validator, the visual-search engine and the model installer. The data-access layer exposes one PhotoCatalogDao interface with two implementations, which is the polymorphism that lets the same controller work against local files or MySQL.

4.2 Database Design

The MySQL mode normalises the catalogue into three tables. A product row carries the curated identity; an offer row carries one retailer's recorded amount together with the provenance needed to re-check it; an image row carries one cached photograph. The primary key of the offer table includes the retailer, which makes a duplicated observation for the same shop impossible at the schema level, and the check constraints reject negative amounts, a currency other than USD and an unknown retailer.

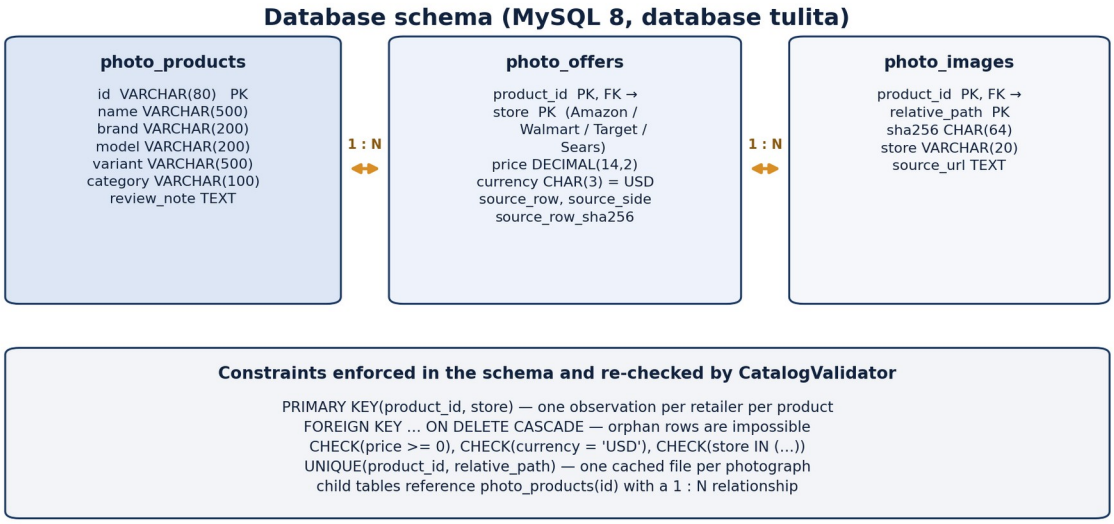


Figure 4.2: Entity relationship diagram of the MySQL schema and the constraints enforced on it

Local-file mode stores the same information in catalog.json, with a separate manifest of hashes. The two modes are therefore representations of one logical model rather than two different systems, and both are re-validated by the same CatalogValidator before being displayed.

4.3 Class Design and Object-Oriented Choices

Class	Responsibility	Object-oriented idea demonstrated
PhotoProduct, Offer, Photo	Immutable records holding a curated variant, one retailer observation and one cached photograph	Encapsulation, immutability, value semantics of records
PhotoCatalogDao	Interface: load and replace the whole catalogue	Abstraction and polymorphism
JsonPhotoCatalogDao, JdbcPhotoCatalogDao	File-backed and database-backed implementations	Interface implementation, dependency inversion
CatalogValidator	Re-verifies hashes, identity and membership rules	Fail-fast validation, defensive programming

Class	Responsibility	Object-oriented idea demonstrated
CatalogService	Search, filtering, sorting, spread and savings arithmetic	Separation of concerns, stateless computation
VisualSearch	Preprocessing, ONNX session, cosine ranking; AutoCloseable	Resource management, try-with-resources
ModelFiles	Pinned download, SHA-256 verification, path resolution	Configuration precedence, integrity checking
CameraSource / WebcamSource	Abstraction over the capture device	Adapter pattern, testability through injection
PhotoCatalogController	Orchestrates load, index, search; owns the worker thread	Concurrency isolation, lifecycle management
PhotoCatalogView, PhotoComparisonDialog, CameraDialog	Presentation only, no data logic	Single responsibility, MVC

Table 4.1: Principal classes and the design ideas they demonstrate

4.4 Design of the Offline Image-Retrieval Path

Retrieval is a five-stage pipeline. The image is decoded, resized so that its longest edge is 224 pixels, padded to a square with white rather than centre-cropped, and normalised with the CLIP mean and standard deviation. The resulting tensor is passed to the ONNX session, which returns a 512-dimensional embedding that is L2-normalised. Cosine similarity is computed against the reference vectors of the cached photographs, the best vector per product is retained, and the five best distinct products are offered to the user.

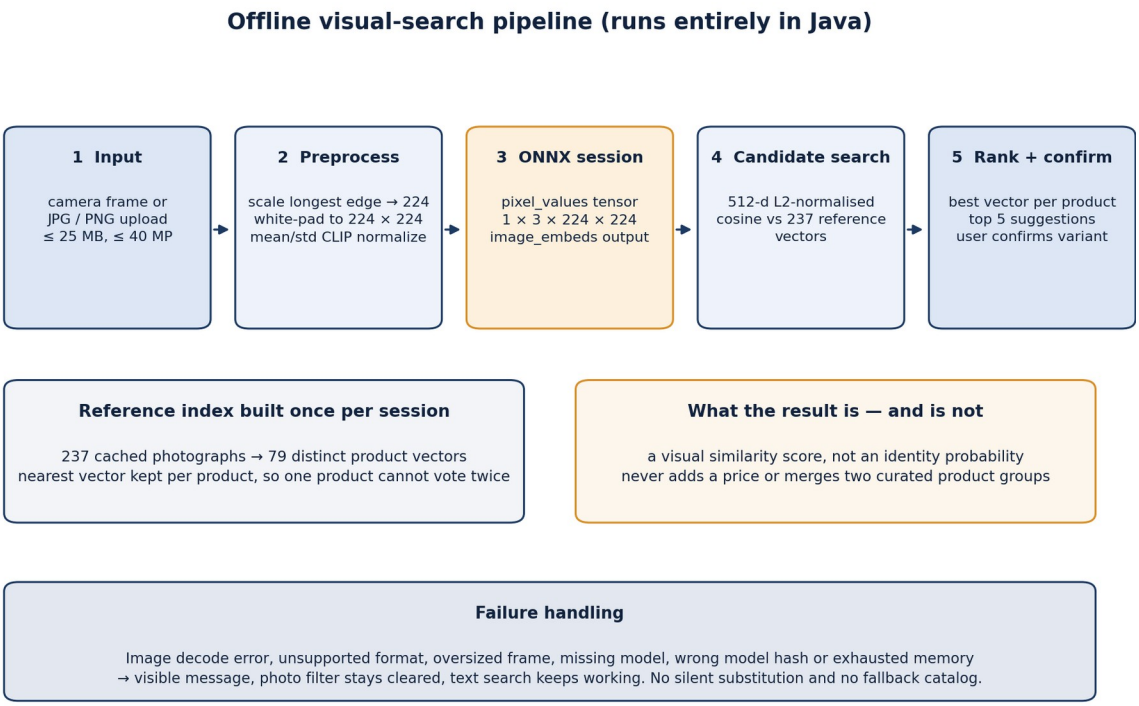


Figure 4.3: The offline visual-retrieval pipeline, including what a suggestion does and does not mean

White padding was a deliberate correction: an earlier centre-crop variant cut the edges off wide packaging such as a microwave carton and demoted the correct product, while padding preserved the whole object. Retaining only the best vector per product prevents a product with many photographs from dominating the ranking, and the confirmation step keeps a similarity score from ever becoming a product decision.

5. IMPLEMENTATION

5.1 Package and Class Layout

The implementation is 90 Java source files across a small number of packages, plus 18 test classes. The principal packages are:

- **tulita** — launcher, application class and mode selection.
- **tulita.photocatalog** — the curated catalogue edition: model records, two data-access implementations, services, views, controller and command-line entry points.
- **tulita.photocatalog.camera** — the CameraSource abstraction and its webcam adapter.
- **tulita.dataset, tulita.matching, tulita.parser** — the earlier India-store editions, retained unchanged as separate modes.
- **src/main/resources** — the JavaFX stylesheet and the database schema copy.

The default mode is the curated catalogue: the application reads the environment, selects the catalogue mode if nothing else is requested, and loads either the JSON catalogue or MySQL according to the configured storage source.

5.2 Data Acquisition and Curation

The historical records come from the ProMap product-mapping dataset, which pairs listings of the same product across retailers and labels each pair. Its four CSV files contain 1,800 labelled pairs, of which 566 carry a positive label and 382 have an amount for both retailers. Positive labels were treated as candidates, not as truth.

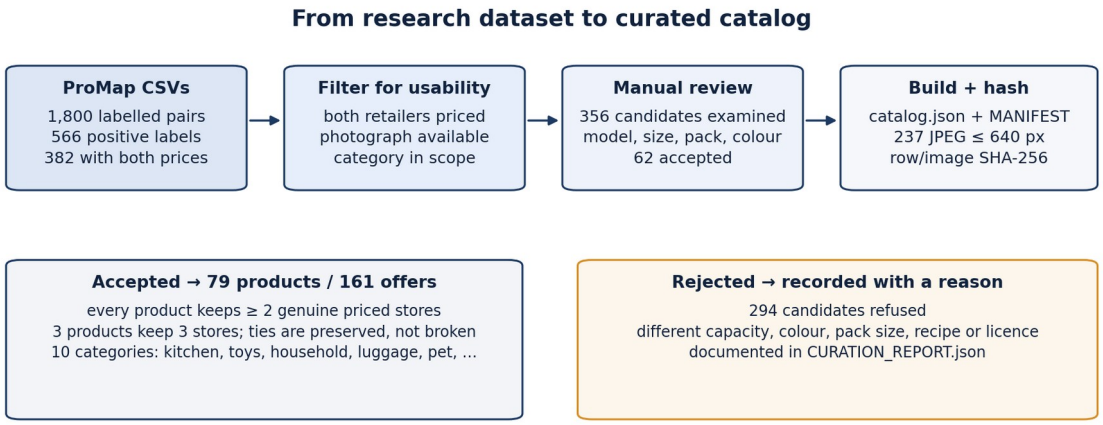


Figure 5.1: From the research dataset to the curated catalogue, showing the accepted and rejected outcomes

Of the 356 candidates that were usable after removing pairs with a missing amount or a missing photograph, 62 were accepted after inspecting both listings, and the remainder were rejected with a recorded reason. Typical rejections were a Farberware 1.1 cu ft microwave paired with a 0.7 cu ft model, a Costway appliance in gold paired with a black and rose-gold version, a Swagtron scooter in pink paired with a blue one, a 42-count capsule pack paired with a ClearMinis pack, and Purina variety packs containing different recipes. Photographs were downloaded only from the image URLs present in the dataset, converted to RGB JPEG with a maximum edge of 640 pixels, and hashed. The result is 79 curated variants, 161 priced observations and 237 cached photographs, with every variant keeping at least two priced retailers. Rejections are listed in CURATION_REPORT.json so that the decision can be audited rather than trusted.

5.3 Local-File Implementation

In local-file mode the catalogue is read from datasets/promap/catalog.json. The loader resolves the data folder in a fixed order — an explicit environment variable, then the working directory, then the folder beside a packaged application — so that the same build works from a project folder, from a terminal and from an installed image. After parsing, every product passes through CatalogValidator, which recomputes the SHA-256 of each source row and of each photograph, checks that the retailer appears in the offer row, checks that two observations of one product do not carry contradictory known model numbers, and rejects any product left with a single priced retailer.

Code excerpt 5.1: Evidence validation before display

```
// CatalogValidator: the checks that gate every displayed amount (reformatted)
for (PhotoProduct p : products) {
    require(p.offers().size() >= 2, "A card needs two or more priced stores: " + p.id());
    for (Offer o : p.offers()) {
        require(STORES.contains(o.store()), "Unknown retailer: " + o.store());
        require(rowHash(p, o).equals(o.sourceRowSha256()), "Source row changed: " + p.id());
        require(p.knownModels().stream().allMatch(m -> o.title().contains(m)),
                "Contradictory model number: " + p.id());
    }
    for (Photo ph : p.photos())
        require(sha(Files.readAllBytes(safe(root, ph.relativePath())))
                .equals(ph.sha256()), "Photograph changed: " + ph.relativePath());
}
```

5.4 MySQL Implementation

The database mode uses the same records but a normalised schema. Loading and importing both run inside a transaction with repeatable-read isolation, so a reader never sees a half-replaced catalogue and a failed insert rolls back completely.

Code excerpt 5.2: Transactional catalogue replacement

```
// JdbcPhotoCatalogDao: transactional replace (reformatted)
try (Connection c = factory.open()) {
    c.setTransactionIsolation(Connection.TRANSACTION_REPEATABLE_READ);
    c.setAutoCommit(false);
    try {
        try (PreparedStatement ps = c.prepareStatement("DELETE FROM photo_products")) {
            ps.executeUpdate();                    // children cascade
        }
        insertProducts(c, products);
        insertOffers(c, products);
        insertImages(c, products);
        c.commit();
    } catch (Exception e) {
        c.rollback();
        throw e;                                   // reported, never hidden
    }
}
```

Credentials are read from a private application.properties file or from environment variables, with the environment taking precedence. No password is present in the source code or in the submitted archive. If the schema is missing, the credentials are wrong or the import has not been run, the interface reports a setup error with instructions; it does not fall back to JSON or to sample data.

5.5 Visual Search and Camera Implementation

VisualSearch owns one ONNX session, created with two intra-op threads, and implements AutoCloseable so that the native session is released deterministically.

ModelFiles pins the model URL and its SHA-256; the download is written to a temporary file, verified and then moved into place, and a file whose hash does not match is rejected rather than used.

Code excerpt 5.3: Cosine ranking with one vote per product

```
// VisualSearch: ranking one query image against the reference photographs (reformatted)
public List<Hit> search(BufferedImage image, int limit) throws Exception {
    if (references.isEmpty())
        throw new IllegalStateException("Reference photo index is not ready.");
    float[] query = embed(image);                    // 512-d, L2-normalised
    Map<String, Hit> best = new HashMap<>();
    for (Reference ref : references) {
        double score = cosine(query, ref.vector());
        Hit before = best.get(ref.product().id());
        if (before == null || score > before.similarity())
            best.put(ref.product().id(), new Hit(ref.product(), score));
    }
    return best.values().stream()
        .sorted(Comparator.comparingDouble(Hit::similarity).reversed()
                .thenComparing(h -> h.product().id()))
        .limit(Math.max(1, limit))
        .toList();
}
```

The camera is reached through the CameraSource interface, whose only production implementation wraps the webcam library. If no device, driver or permission is available, the dialog disables capture and points the user to the upload path; a synthetic frame is never substituted. The design also allows test providers to be injected, which is how the success and failure flows were tested without physical hardware.

5.6 User-Interface Implementation

The interface is a single scrollable workspace with a navy side rail, a headline band of four counters, a toolbar of filters and a grid of product cards. Each card lists the recorded amount of every retailer for that variant, marks the lowest one, and shows either a spread badge or a tie badge. The comparison sheet presents four key figures, a bar chart of the recorded amount per retailer, an evidence table and the source photographs. Styling is entirely local CSS with system fonts, so no network asset is needed and the window renders identically offline.

Two interface decisions are worth stating explicitly because they concern honesty rather than appearance. The comparison sheet labels the smallest amount “lowest recorded item amount” and not “best price”, because delivery, stock and currency conversion are not in the source. The rating column is present but reads “Not in source”, because the source

files contain no rating field; an empty star rating was rejected as more misleading than an explicit absence.

6. EXPERIMENT RESULTS AND ANALYSIS

6.1 Test Environment

Item	Value
Operating system	Linux x86-64, headless server with Xvfb 1600 × 1200
Java runtime	OpenJDK 21.0.12.1, JavaFX 21.0.6
Build tool	Apache Maven 3.9.9, Surefire 3.5.2
Database	MySQL 8.0.40, database tulita_review_test for destructive tests
Inference	ONNX Runtime 1.20.0, CPU execution provider
Catalogue under test	79 products, 161 offers, 237 photographs
Verification date	19 October 2026

Table 6.1: Environment used for the reported measurements

6.2 Catalogue and Price Analysis

The curated catalogue is summarised in Table 6.2. Sixteen category labels are in use, covering kitchen appliances, toys and games, household goods, clothing and gear, pet supplies, bags and travel, health, home décor, garden and outdoor, arts and crafts, electronics, computers and phone accessories. Amazon and Walmart appear for all 79 variants; Target appears for the three variants where a Target observation passed review.

Measure	Value
Curated product variants	79
Priced retailer observations	161
Cached product photographs	237
Distinct brands	72
Variants with exactly 2 priced retailers	76
Variants with 3 priced retailers	3 (Hamilton Beach 58149, Reynolds Wrap, Franklin dartboard)
Exact ties between retailers	12
Total difference between lowest and highest recorded amount	USD 1,290.06 across all 79 variants
Median difference as a share of the higher amount	17.6 %

Measure	Value
Variants with a difference above 10 % / 20 % / 30 %	53 / 36 / 23
Largest absolute difference	USD 297.25 (RCA RDW3208 dishwasher, 232.74 vs 529.99)
Largest relative difference	71.5 % (Dagamma DP100 pulse oximeter, 9.97 vs 34.95)
Smallest non-zero difference	USD 0.04 (LEGO Bookshop 10270, 199.95 vs 199.99)

Table 6.2: Catalogue composition and price-spread statistics

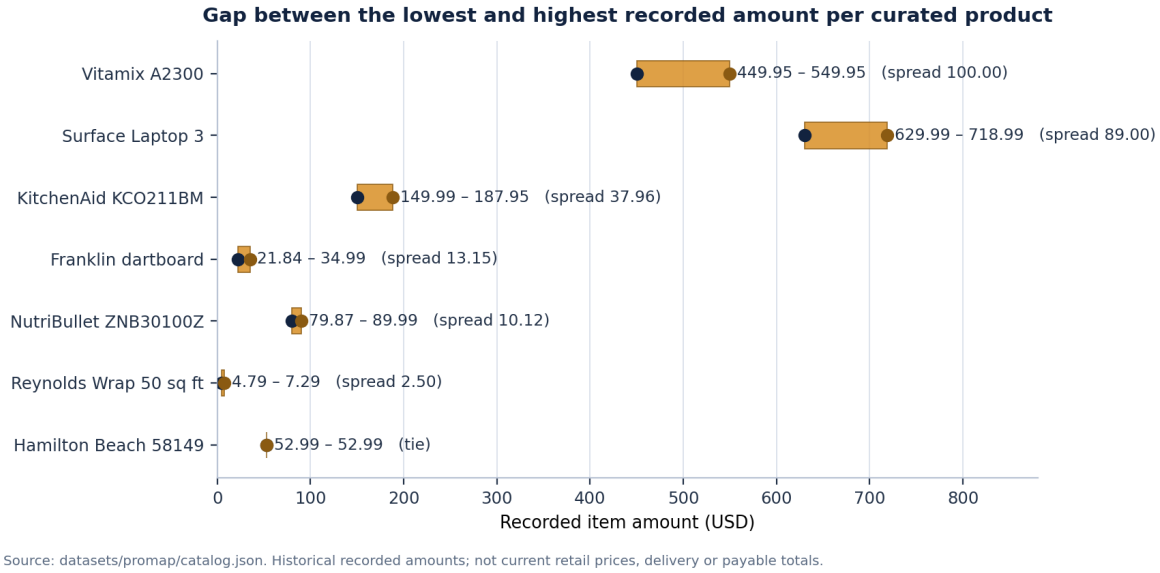


Figure 6.1: Difference between the lowest and highest recorded amount for selected curated products, computed from the recorded observations

Two conclusions follow from these figures. First, the spread is not a rounding artefact: the median variant is 17.6 % cheaper at one retailer than at the other, and twenty-three variants differ by more than 30 %. Second, and more importantly for an honest project, twelve variants are exact ties, and the application reports them as ties instead of declaring a winner. Hamilton Beach 58149 is recorded at USD 52.99 by all three retailers, which is a genuine three-way equality in the source data.

The RCA dishwasher pair illustrates why validation matters. The two recorded amounts differ by USD 297.25, a 56 % gap, which is plausible for a countertop dishwasher but would also be the expected signature of a mismatched pair. The pair was accepted only after the model number RDW3208 was found in both listings; the review note records that basis, so a reader can disagree with the decision and re-check the row.

6.3 Comparison Results

Figure 6.2 shows the main workspace with the full catalogue loaded. The headline counters report 79 products, 161 recorded prices, 237 photographs and 3 three-store groups, all computed from the loaded data rather than hardcoded.

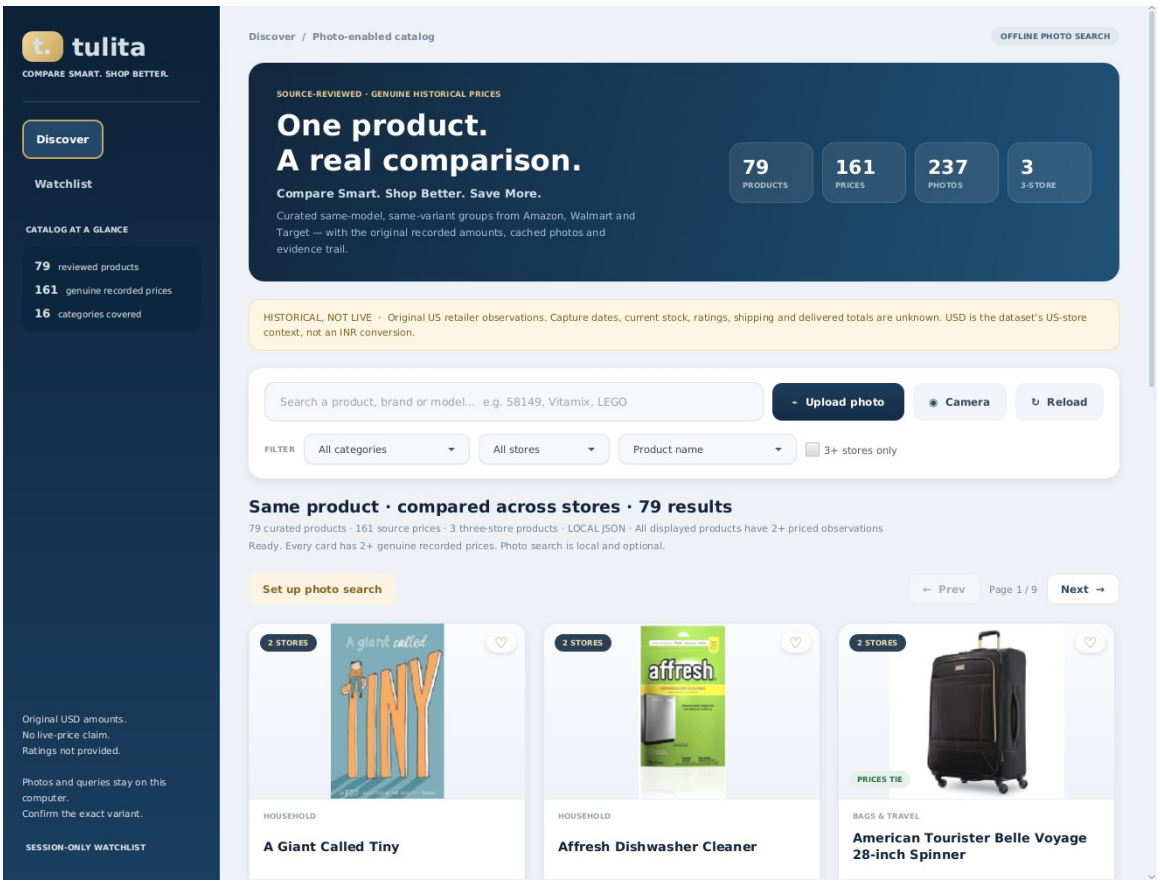


Figure 6.2: Browsing workspace with the catalogue loaded; the counters are derived from the validated data

Figure 6.3 shows the comparison sheet for the Franklin Sports electronic dartboard, a three-retailer group. The recorded amounts are USD 21.84 at Walmart, USD 34.99 at Amazon and USD 23.99 at Target, giving a spread of USD 13.15, or 37.6 % below the highest amount. The evidence table beneath the chart lists each observation separately, and its rating field reads “Not in source” because the source files contain no rating.

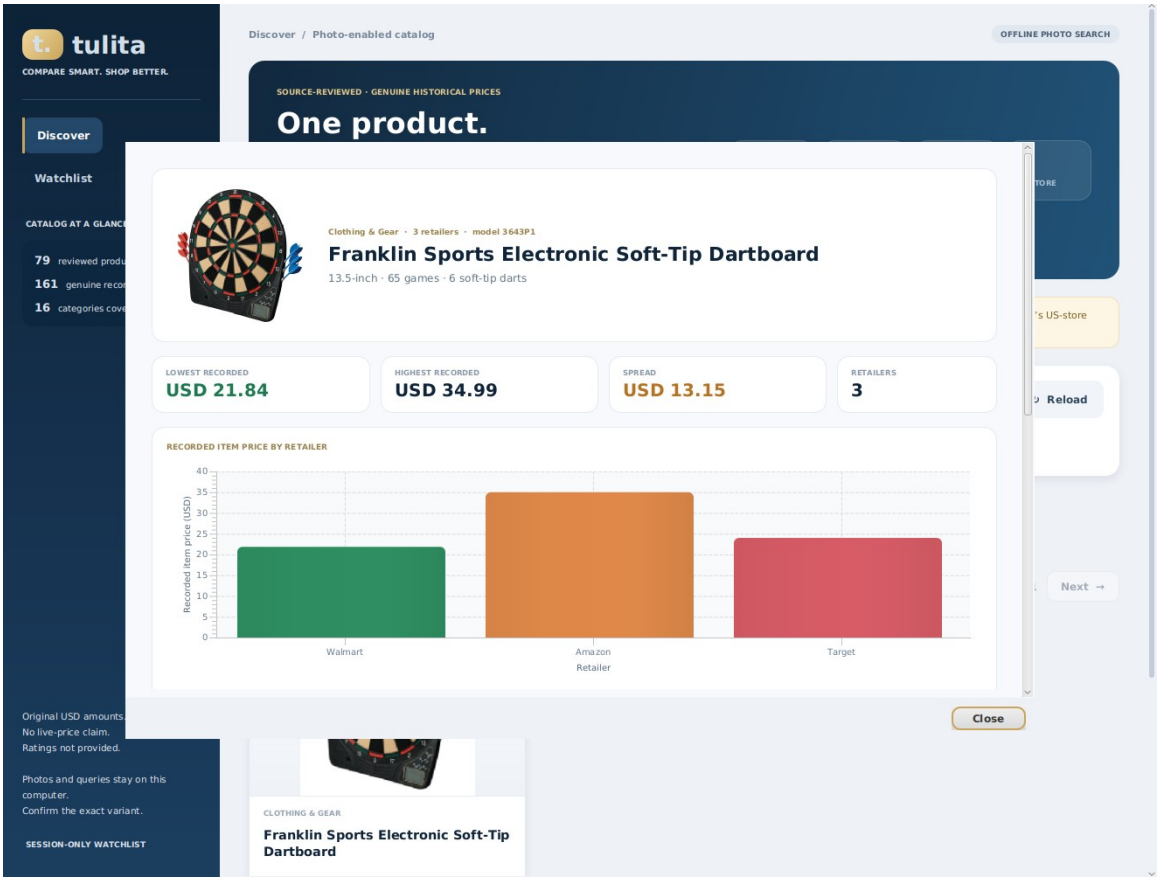


Figure 6.3: Comparison sheet for the three-retailer Franklin dartboard group, with the key figures, the recorded amount per retailer and the evidence table

Figure 6.4 shows the opposite outcome. The Hamilton Beach Power Elite blender and chopper, model 58149, is recorded at USD 52.99 by Walmart, Amazon and Target alike. The sheet reports a spread of USD 0.00 and highlights no winner, because declaring one retailer cheaper would misrepresent the records. A comparison system that can display a tie credibly is a stronger result than one that always finds a bargain, and this case is used deliberately during the demonstration.

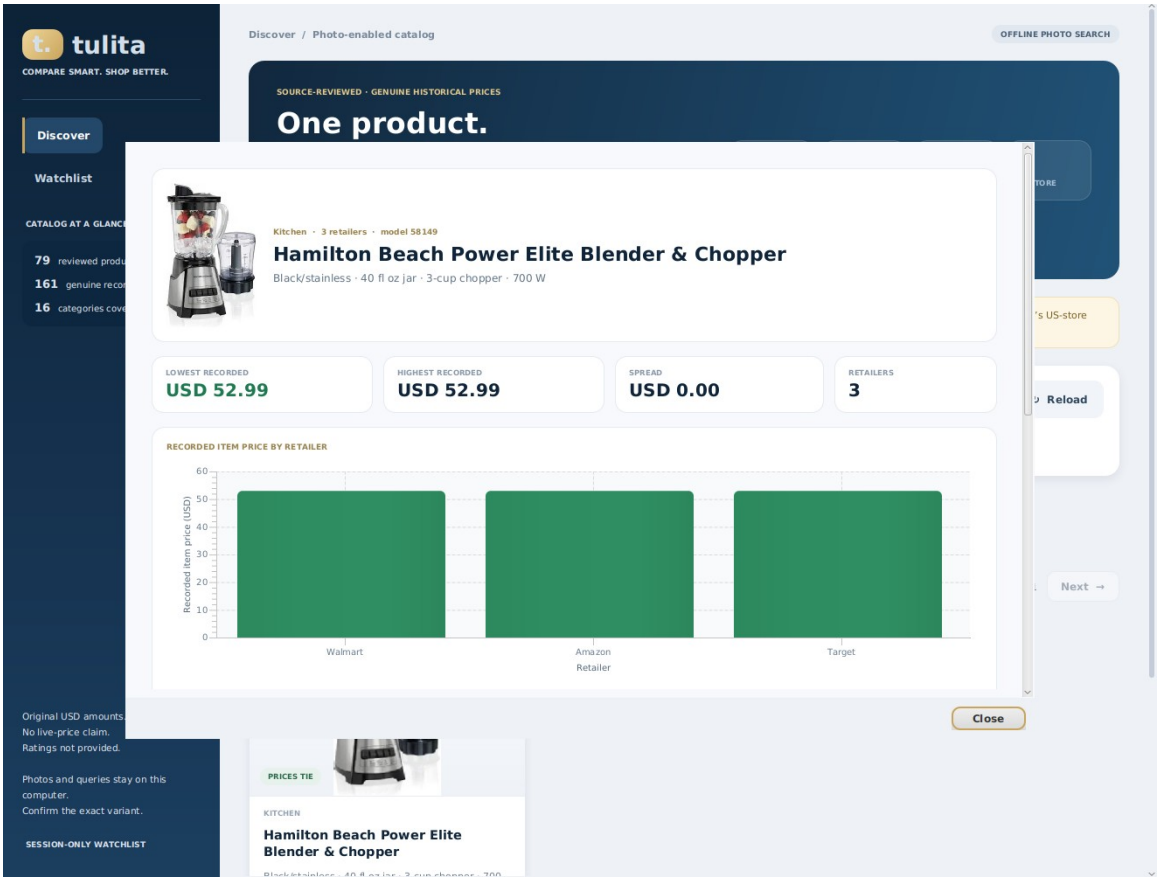


Figure 6.4: A genuine three-way tie: identical recorded amounts produce a spread of USD 0.00 and no highlighted winner

6.4 Image-Search Results

Held-out photographs were used as queries: for three curated products, a photograph of the same product taken from a different retailer was supplied as the query image and the correct product was ranked first in each case, as recorded in the verification file. This is a functional check that the pipeline is wired correctly, not an accuracy benchmark, because the catalogue holds only 79 products and the query images come from the same source collection.

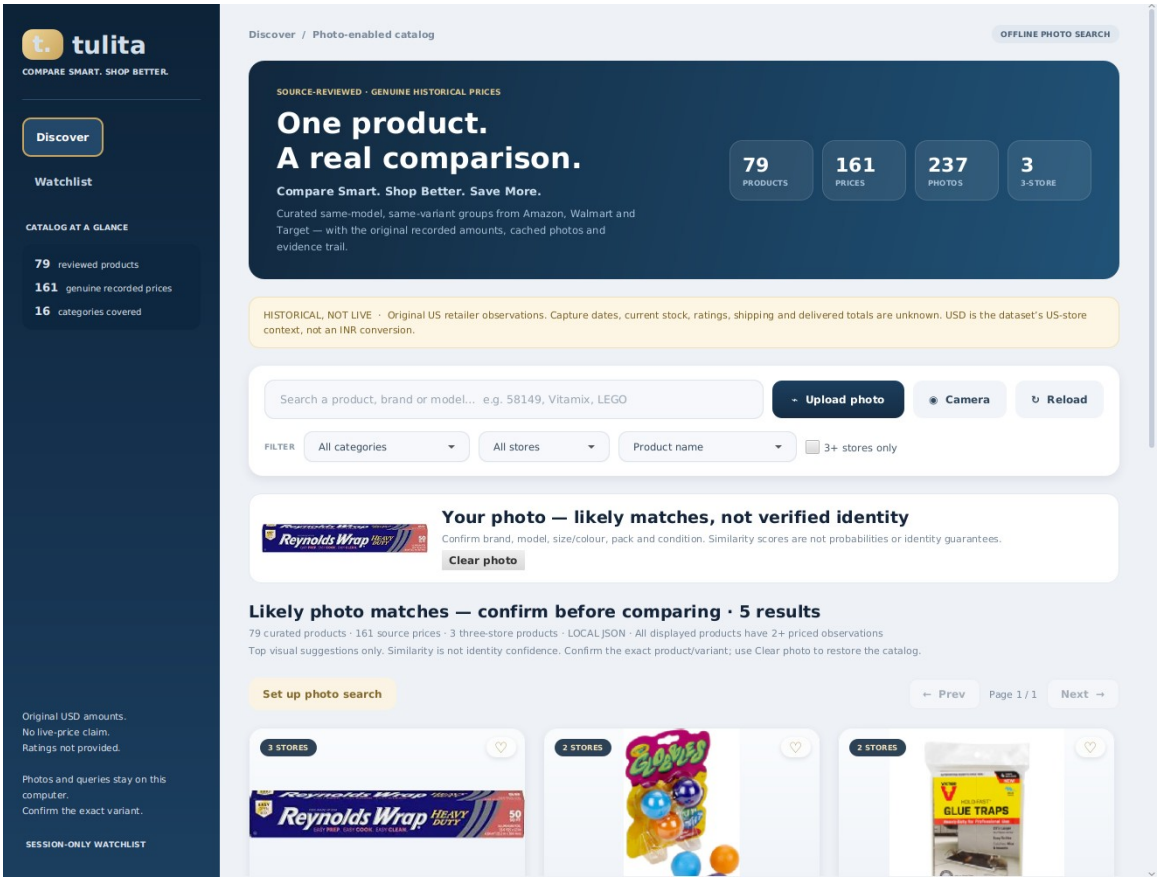


Figure 6.5: Photo-based retrieval; the suggestions are ranked candidates and the user must confirm the variant

Deliberately, an incorrect suggestion has no consequence beyond an extra click: the suggestion does not open a comparison, does not add a retailer and does not modify the catalogue. The failure path was also exercised — an unsupported file, an oversized image and a missing model each produce a visible message while text search continues to work.

6.5 Automated Test Results

Test class	Scope	Result
PhotoCatalogTest	Catalogue loading, validation, search, spread arithmetic, rejection of altered evidence	Passed
PhotoVisionTest	ONNX session, preprocessing, embeddings, ranking, model hash (opt-in)	Passed
PhotoCatalogUiTest	JavaFX view construction and interaction under a display, screenshot capture (opt-in)	Passed
PhotoCatalogMySqlTest	Schema, transactional import, reload, rollback on failure (opt-in, destructive)	Passed
BudgetOptimizerTest	Planner engine, scoring, cost arithmetic, charges, what-if, persistence, currency (28 tests)	Passed

Test class	Scope	Result
BudgetOptimizerUiTest	Planner inside the running window: navigate, optimise, what-if, per-line charges, save and re-open, printable plan, infeasible and invalid input (opt-in)	Passed
Dataset, MultiStore, SameProduct suites	Earlier editions retained as separate modes	Passed
Legacy UI and MySQL suites	Earlier editions retained as separate modes	Passed

Table 6.3: Test suite coverage and outcome

The complete suite reports 133 tests with 0 failures and 0 errors. In a default mvn test run, 122 of them execute and 11 optional tests are skipped, because the desktop, model and destructive database tests require a display, the downloaded weights and a dedicated review database respectively. All 133 pass when those prerequisites are supplied. On a machine with about 2 GB of memory the JavaFX test class is run separately from the database classes, because a single forked JVM running everything is killed for memory; this is an environment limit recorded here rather than hidden.

Packaging was verified as well. The jpackage image was built, copied to an unrelated folder and launched without Maven or an installed JDK. The application resolved its catalogue, displayed all 79 products with photographs, and resolved the bundled model path with a matching SHA-256, as shown in Figure 6.6.

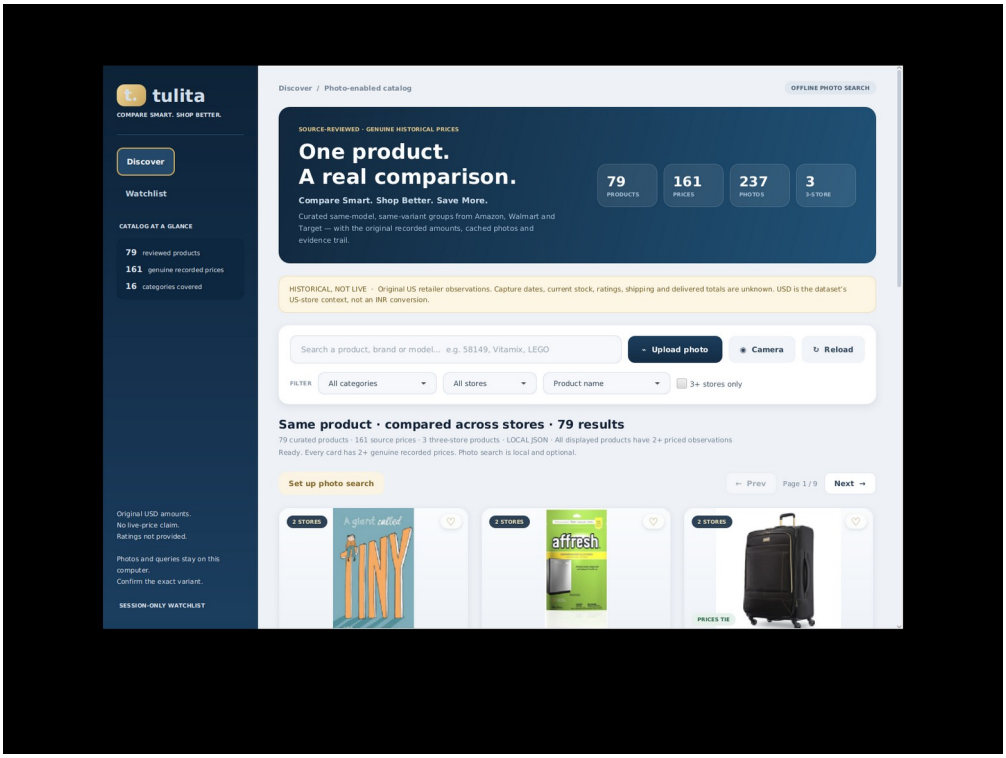


Figure 6.6: The self-contained application image running from an unrelated directory with no JDK or Maven installed

6.6 Discussion and Limitations

The results support four claims and no more than four:

- Every comparison displayed is between observations that were individually checked to be the same variant, and each is traceable to a source row by hash.
- The amounts are genuine historical records; they are not current prices, and the interface says so.
- The application works identically from local files and from MySQL, with no silent fallback between the two.
- Photo retrieval works fully offline inside the Java process.

The following limitations are stated explicitly and are not hidden in the interface:

- **No ratings or reviews.** The four source files contain no rating or review-count column and no rating specification key, so the interface shows “Not in source” rather than a value.
- **No price history.** Each retailer supplied one undated observation per product, so a trend over time cannot be drawn from this data. The chart shows the recorded amount per retailer instead; a peak-and-fall curve would have to be invented and was therefore not drawn.

- **No live amounts.** Nothing is refreshed from a retailer site. Retailer links are retained as provenance and may have expired.
- **Currency assumption.** The source has no currency column; USD is inferred from the United States retailer context and is documented as an assumption, not as a recorded fact.
- **Coverage.** The catalogue holds 79 variants, so text search is a search of a finite curated catalogue, not of the whole retail web.
- **Camera and Windows.** No physical webcam was available in the verification environment; capture flows were tested with injected providers. The Windows build script follows documented jpackage behaviour but was not executed on Windows here, and phone deployment is out of scope because JavaFX is a desktop toolkit.
- **Model caveat.** Visual similarity is not an identity probability. A product outside the catalogue can still have a nearest neighbour, which is exactly why the user confirms the variant.

7. INTELLIGENT BUDGET OPTIMIZER

The comparison feature answers “how much does this product cost at each retailer?”. The budget planner answers the question a shopper actually starts with: “given the money I have, what should I buy?”. It is a second use of the same curated evidence rather than a second application. The module lives in the tulita.budget package tree, is opened from the same JavaFX side rail as the catalogue, and reads the same PhotoProduct and Offer records that the comparison screen validates. No product model was duplicated, no database table was added and no dependency was introduced; pom.xml is unchanged in this round.

7.1 Requirement Analysis for the Planner

The planner was specified as an optimisation problem with explicit honesty constraints. Table 7.1 lists the requirements and the mechanism that satisfies each one; the remainder of the chapter explains the mechanisms that carry real design weight.

ID	Requirement	How it is satisfied
PR1	Enter a total budget; plan in INR if required	BudgetMoney parses and scales to two decimals; CurrencyConverter applies a rate the user declares, or keeps the catalogue currency
PR2	Required and optional categories with quantities	BudgetRequest.CategoryRequirement: required flag, min/max quantity, brand, priority, minimum specification, target unit price
PR3	Recommend by price, specifications, brand and value for money	ProductScoringService with five weighted components over catalogue-derived metrics
PR4	Two distinct modes	RecommendationMode: Best Value and Lowest Cost, plus Best Specifications and Balanced; all four read one Pareto front
PR5	Never exceed the budget; never relax a mandatory rule	Hard constraint inside the engine plus OptimizationConstraintValidator, which re-checks the basket before anything is displayed
PR6	Dashboard of items, prices, total, remaining, utilisation and category spend	BudgetBreakdown plus a JavaFX BarChart that redraws for every result
PR7	Explanations and alternative combinations	RecommendationExplanationService; alternatives are de-duplicated by basket signature
PR8	What-if re-optimisation	WhatIfOption builds complete alternative requests; RecommendationDelta reports what moved
PR9	Use only recorded prices; fabricate nothing	Candidates are built from validated offers only; no shipping, discount, rating or availability value is invented — the interface labels the user's own assumptions
PR10	Save and export	JSON history behind a DAO interface, CSV through Commons CSV, and printable output through the JavaFX print pipeline

Table 7.1: Planner requirements and the mechanism that satisfies each

7.2 Module Architecture

The module repeats the layering already used by the catalogue edition, so that a reader who understands one part of the project understands all of it. Dependencies point downward only: the view never touches a repository, the engine never touches JavaFX, and every class that performs arithmetic is testable with no display and no network.

Layer	Classes	Responsibility
model	BudgetRequest, BudgetRecommendation, BudgetBreakdown, SelectedProduct, ProductCandidate, ProductScore, RecommendationMode, PreferenceWeights, ScoringCriterion, RecommendationAlternative, LineCharges, RecommendationDelta, WhatIfOption, OptimizationResult, SavedBudgetRun	Immutable records; structural validation only
dao	BudgetHistoryDao, JsonBudgetHistoryDao	Saved recommendations as JSON under ~/.tulita, overridable by TULITA_BUDGET_STORE; products are stored by reference
service	BudgetOptimizerService, ProductScoringService, CategorySpecProfiles, BudgetCalculationService, RecommendationExplanationService, BudgetExportService, BudgetMoney, CurrencyConverter	Orchestration, scoring, cost arithmetic, explanations, export, money and currency
engine	BudgetOptimizationEngine, OptimizationConstraintValidator	The search itself and the independent self-check of its output
controller	BudgetOptimizerController	Reads the form, runs the optimiser on a daemon worker, hands results back with Platform.runLater
view	BudgetOptimizerView	Four-step form, dashboard, charts, product cards, alternatives, what-if panel, saved plans; no business decisions

Table 7.2: The budget module, layer by layer

The split between service and engine is deliberate. The engine knows only about candidate options, integer costs and scores, so it can be exercised by a unit test that constructs numbers directly; the service knows about catalogues, categories and money formatting, so it can be tested against the real 79-product catalogue. Neither class creates a JavaFX node, and the view cannot compute a price.

7.3 The Optimisation Algorithm

The problem is a multi-choice knapsack with retailer-dependent costs. For each selected category the shopper must buy a quantity q in [qmin, qmax] of one product, and the cost of a basket is not the sum of item prices alone, because a delivery fee is charged once per retailer and a discount may apply to the item subtotal. A greedy sort by price is therefore wrong in both directions: it can miss a cheaper combination that consolidates retailers, and it cannot express a weighted preference at all.

The engine formulates the search over states of the form (spent cents, retailer mask, weighted score sum, specification sum, chosen lines). For each category it generates the small set of candidate options: every eligible product at every permitted quantity, plus the empty option when the category is optional. States are expanded category by category and pruned by Pareto dominance — a state is discarded when another state has already spent no more, bought at least as much score, used no more retailers and reached the same category. Because money is carried in integer cents, comparisons are exact and the result is reproducible.

Code excerpt 7.1: Pareto-pruned dynamic programme used by BudgetOptimizationEngine

```
states <- { empty }
for each required/optional category c in a fixed order:
    next <- {}
    for each state s in states:
        for each option o of c that keeps s.spent + o.cost <= budget:
            next <- next + { s extended by o }
    states <- pareto_prune(next)          # cost up, score up, retailers down
    if |states| > MAX_STATES:             # 20 000
        states <- trim(states); exhaustive <- false
best <- argmax over states of comparator(mode)
return validator.recheck(best, request)   # throws instead of displaying a wrong plan
```

Complexity is O(C · S · F), where C is the number of selected categories, S the number of surviving states and F the average option count per category. With the curated catalogue the fronts stay in the tens, so a full optimisation over 79 products completes in 13–110 ms on the verification machine. When the state cap binds, the engine records exhaustive = false and both the interface and the CSV export describe the result as a bounded heuristic; no global-optimality claim is made in that case.

The four modes are comparators over the same front rather than four separate searches, which guarantees that they cannot disagree about feasibility. Best Value maximises the weighted score sum and then prefers the cheaper basket; Lowest Cost minimises the total

and then prefers the higher score; Best Specifications maximises the specification sum; Balanced maximises 0.6 × mean product score + 0.4 × the share of the budget left unspent. Determinism is protected by sorting categories, candidates and options explicitly and breaking every tie with a stable signature, so identical input always produces the identical basket.

7.4 The Scoring Model

A recommendation is only explainable if the score behind it is stated. Each candidate receives five components, normalised inside its own category so that a USD 12 toy and a USD 900 laptop are never compared directly, and combined with the weights the shopper sets on the sliders.

Component	Measurement	Default weight	Missing data
Price	min–max inverted inside the category; target/price capped at 1 when a target price is set	30 %	never missing: a candidate without a usable price is excluded
Specifications	0.7 × mean of present normalised metrics + 0.3 × attribute coverage	30 %	neutral 0.5, disclosed in the limitations list
Value for money	specification score per unit price, min–max normalised	20 %	neutral 0.5, disclosed
Brand	1.0 on a match, 0.0 otherwise	10 %	criterion dropped when no brand is requested
Warranty	normalised warranty length	10 %	criterion dropped and stated when the category records no warranty

Table 7.3: Score components, weights and missing-data policy

Score = Σ (normalised weight × component) / Σ (normalised weight) over the criteria that survive, so a category with no warranty data is scored on the criteria it does have and the omission is reported instead of being guessed. Two figures are shown to the user: the plain average score per item, and the weighted score for the stated priorities, which is the quantity Best Value actually maximises. Showing only the average would let a five-item Best Value basket look worse than a three-item Lowest Cost basket, which would read as a contradiction.

Two rules keep the score defensible. First, specification metrics are merged per curated variant before scoring, because both retailers describe the same physical product; a shop that documents more specification keys therefore cannot make its own price look better

than the other shop's price for the same item. Second, units are converted rather than guessed: a “180-day limited warranty” becomes 6 months, “1 year” becomes 12 months, and GB/MB, oz/qt/l and lb/kg are converted by the same rule. Both rules are covered by tests, because both were originally wrong.

7.5 Money, Charges and the Rupee Budget

Every monetary value is a BigDecimal with scale two and HALF_UP rounding; the search converts to integer cents so that the budget comparison is exact and the dynamic programme is reproducible. The total is defined as item subtotal + delivery charged once per retailer − discount, where the discount applies only when the item subtotal reaches its threshold. Charges are attributed back to individual lines by BudgetCalculationService.allocate: delivery is divided between that retailer's own lines in proportion to value, the discount is divided across all lines in proportion to value, and any rounding remainder goes to the largest line, so the per-line final costs add up to the basket total to the cent. A UI test asserts exactly that identity, and the identity is what allows a product card to show its own final cost honestly.

The catalogue records its amounts in a single currency — United States dollars for the bundled data — and contains no exchange rate, and the project is not permitted to invent one. The planner therefore separates the currency of the data from the currency on screen. Step 1 offers a currency selector; choosing INR reveals a rate box, and the user supplies the rate to apply (for example 83.50 rupees per dollar). CurrencyConverter then converts at exactly two edges: the budget the user types is converted into the catalogue currency before it reaches the optimiser, and amounts are converted for display, export and printing. The engine, the arithmetic, the hard budget constraint and everything persisted stay in the recorded currency, so a rate can never change which basket is selected. Conversion is monotonic, so a basket inside the budget cannot appear above it in rupees. A blank, zero, negative or non-numeric rate is rejected with a message that names both currencies, and the typed budget is echoed back exactly as entered rather than with conversion round-trip noise. Amounts are printed with Indian digit grouping, so a hundred thousand rupees reads 1,00,000.00.

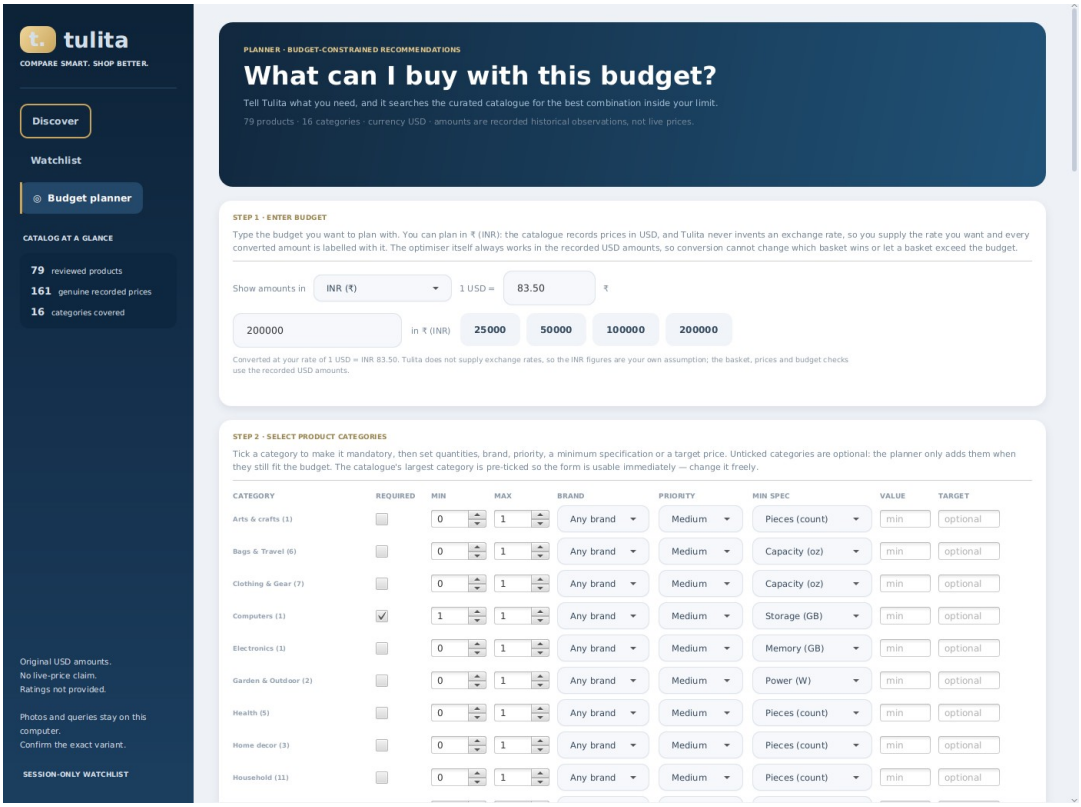


Figure 7.1: Step 1 with the rupee budget: the user's own rate is disclosed, and the form echoes the amount exactly as typed

7.6 The Planner Interface

The planner is a single scrollable workspace reached from the rail entry “Budget planner”, and it keeps the navy-and-amber language of the catalogue. It is organised as four numbered steps — budget, categories, preferences, optional charges — followed by one primary action, “Optimize My Budget”, so the user can always see what the optimiser was told. Category rows carry their real product counts, required/optional state, minimum and maximum quantity, brand, priority, minimum specification and target price; the largest catalogue category is pre-ticked so the form is usable immediately.

The result screen is a dashboard rather than a list. Four KPI tiles report the total budget, the estimated total, the remaining balance and the utilisation percentage; a bar chart shows spending by category; and each product card carries the photograph, brand and model, category and retailer, unit price, quantity, its own share of delivery and discount, the final cost, the recorded specifications, the observation timestamp, a link to the recorded retailer page and the TULITA Recommendation Score, followed by the sentence that explains why it was chosen and the trade-off against the cheapest eligible alternative in that category. Alternatives from the other objectives are listed only when they are genuinely different, and identical baskets are never repeated under another name.

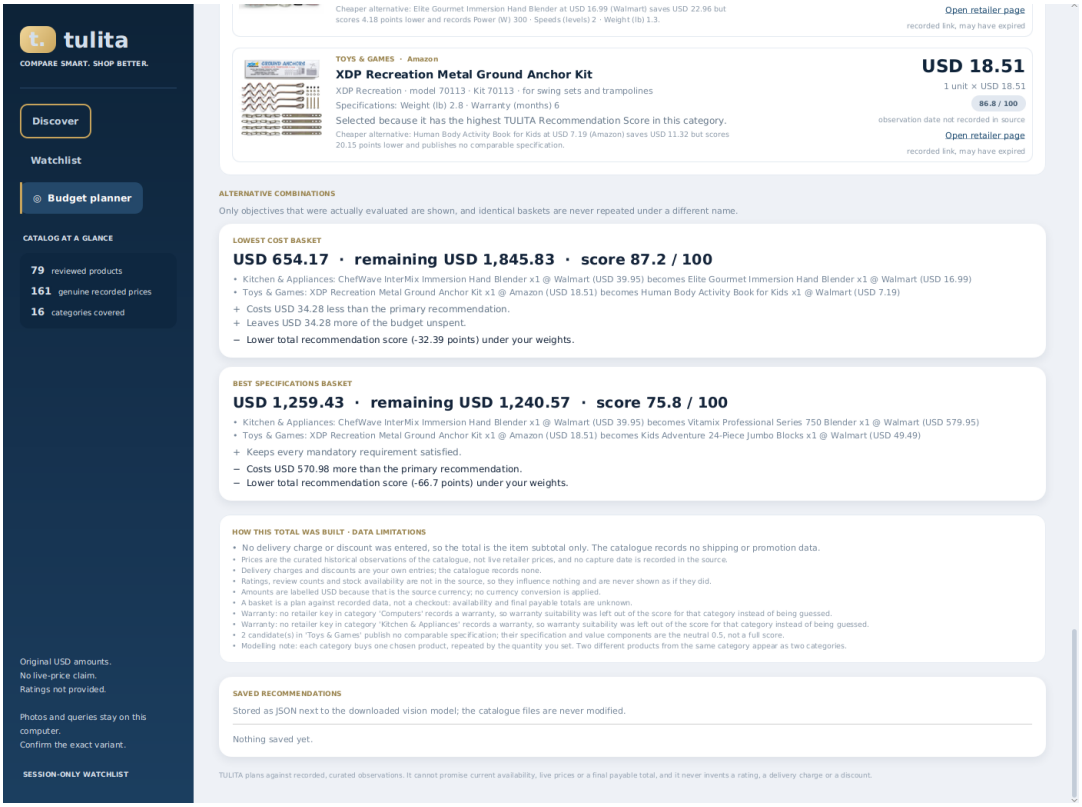


Figure 7.2: The recommendation dashboard: KPIs, spending by category, explained product cards and the trade-off against the cheapest eligible alternative

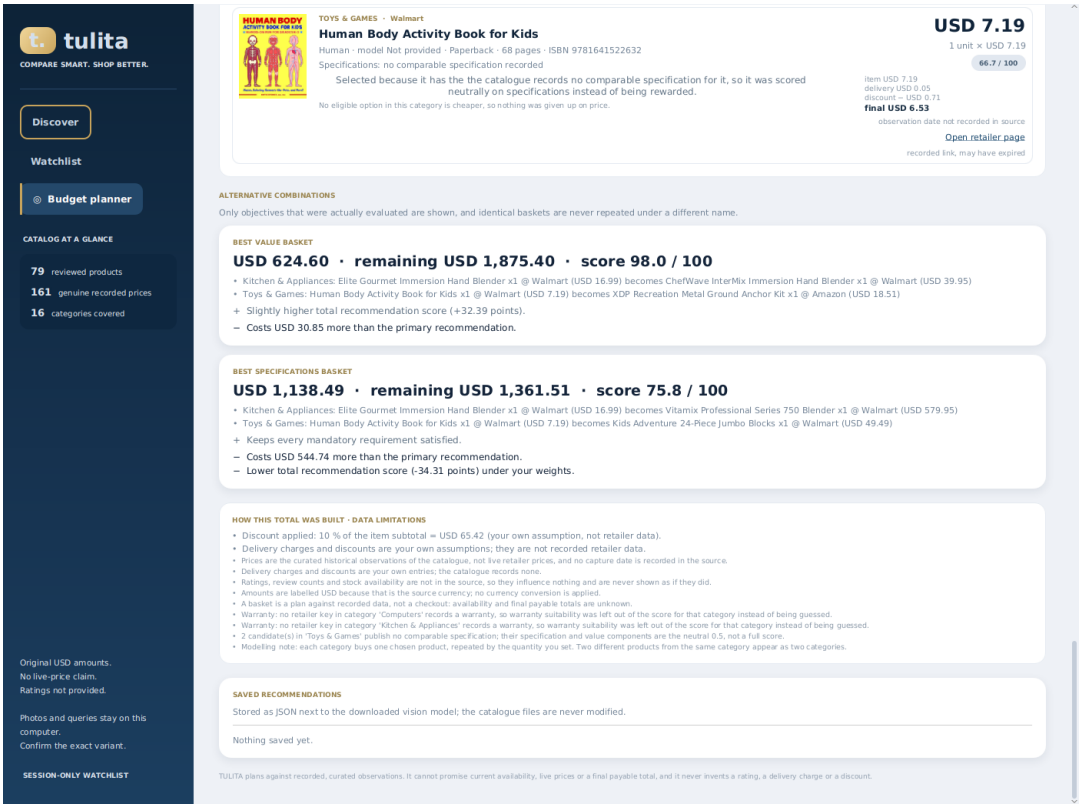


Figure 7.3: A card closed by its own item, delivery, discount and final figures beside the alternative combinations that were actually evaluated

7.7 What-If Analysis

A recommendation is a starting point, so the planner makes the counterfactual cheap to test. The Explore Alternatives panel offers ready-made requests: ten per cent less or more money, each other mode, a mandatory category made optional, one unit fewer, any brand, a relaxed minimum specification, and removing the user's own delivery and discount assumptions. Each option carries a complete request rather than a hidden modification, so choosing one writes the whole request back into the visible form, re-runs the optimiser on the background worker thread and then reports the comparison in a dedicated card: the signed change in estimated total, the change in remaining money, the change in score, the item count, and each product that was swapped, with both amounts. If the basket genuinely does not change, the card says so and states that the outcome is a real result rather than a cached one.

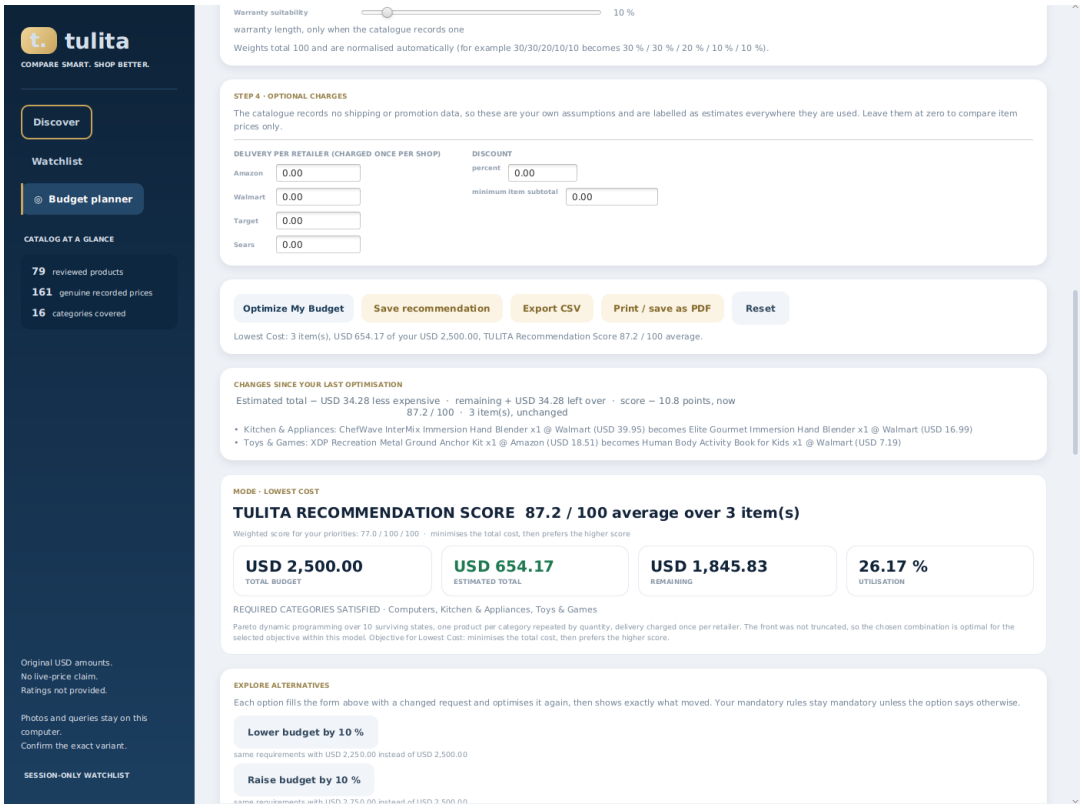


Figure 7.4: A what-if run after switching to Lowest Cost, with the change card above the new recommendation

7.8 Impossible Requests and Failures

The planner never relaxes a mandatory rule to manufacture a result. When no combination satisfies the request it reports infeasibility, states the binding reason in the language of the constraint that failed, and lists the cheapest basket that would satisfy the

mandatory categories together with the difference from the budget the shopper entered. Figure 7.5 shows the response to a 40-dollar request when the cheapest conforming basket costs 654.17 dollars; the screen names the shortfall instead of showing an empty list.

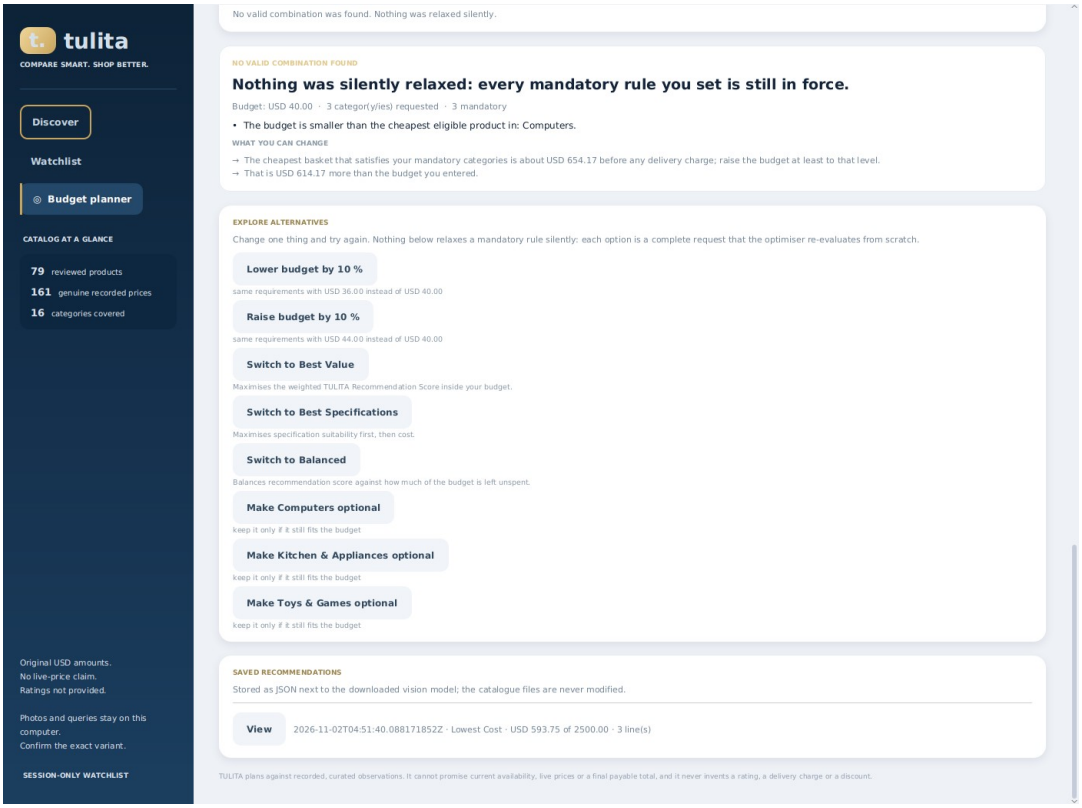


Figure 7.5: An impossible request is explained, and the cheapest conforming basket is quoted

Validation happens before the search as well. A negative budget is rejected by the record's own constructor, a zero budget is reported as a structural error, an unparseable amount names the text that failed, and a quantity range whose minimum exceeds its maximum is refused. The optimiser also re-checks its own output: every mandatory category must be present, quantities must lie inside the requested range, minimum specifications must hold, and the total must be at or below the budget. A violation throws rather than rendering a wrong plan.

7.9 Results

Table 7.4 reports the four modes on the real catalogue with a budget of 2,500.00 USD and three mandatory categories (one computer, one to two kitchen appliances, one to two toys), measured on the verification machine. The modes read one Pareto front, so their differences come from the objective and not from a different search.

Mode	Estimated total	Remaining	Utilisation	Authored score	Items	Time
Best Value	USD 746.91	USD 1,753.09	29.88 %	78.1 / 100	5	110 ms
Lowest Cost	USD 654.17	USD 1,845.83	26.17 %	82.6 / 100	3	29 ms
Best Specifications	USD 1,888.87	USD 611.13	75.55 %	57.2 / 100	5	13 ms
Balanced	USD 654.17	USD 1,845.83	26.17 %	82.6 / 100	3	13 ms

Table 7.4: The four modes on the curated catalogue (budget USD 2,500.00)

Two readings of that table are worth stating, because a careless report would present only one. Best Value spends more than Lowest Cost and buys more items, which is what a weighted objective should do; its plain average score is lower while its weighted score is higher, which is exactly why both figures are shown. Balanced coincides with Lowest Cost here because the unspent-budget term dominates in this test case, and the report says so rather than implying a fourth distinct policy. The planner also measured a rupee budget end to end: 2,00,000 rupees at a user rate of 83.50 produced a catalogue budget of USD 2,395.21, an estimated total of INR 54,623.20, a remaining balance of INR 1,45,376.84 and a utilisation of 27.31 %, with the rate disclosed beside the figures.

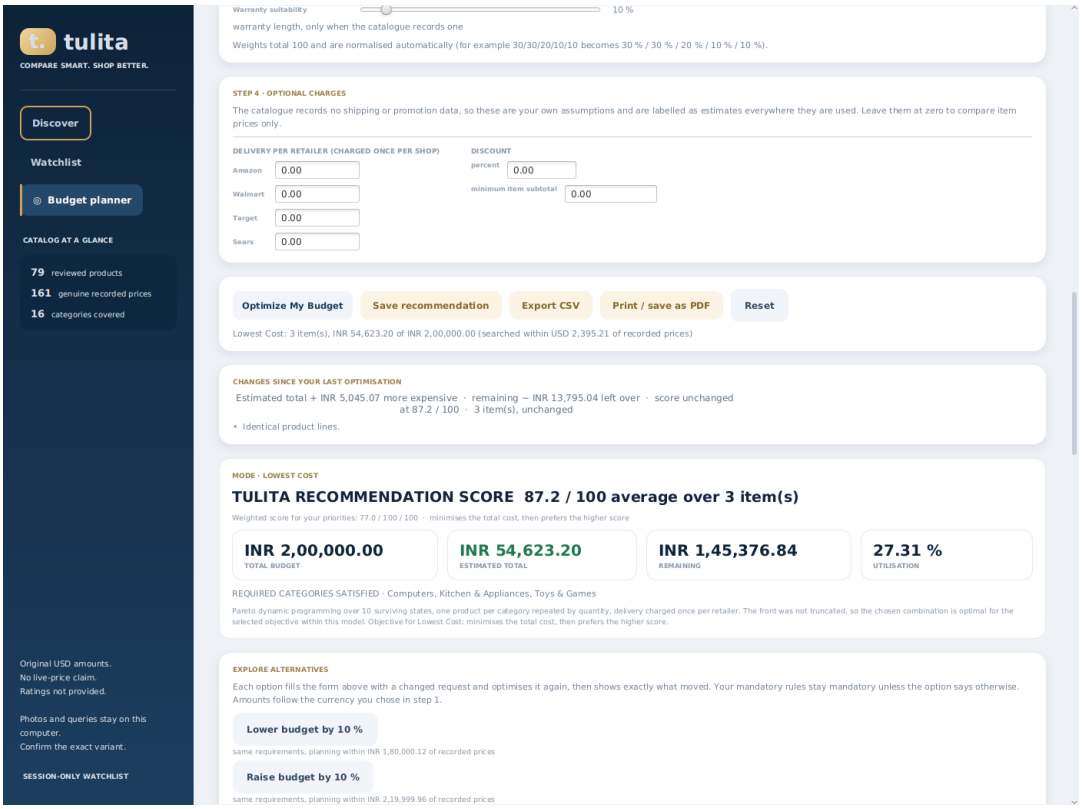


Figure 7.6: The same dashboard in rupees, including the rupee budget, total, remaining balance, utilisation and category spend

7.10 Automated Tests for the Planner

Twenty-eight unit tests cover the planner without a display, a database or the network, and one end-to-end UI test drives the real JavaFX window. Table 7.5 groups them by the risk they address; the full suite now reports 133 tests, 122 of which execute in a default run with 11 optional tests skipped and no failures.

Area	Tests	What is asserted
Budget limits	1, 2, 13	a valid budget produces a feasible plan and never exceeds it; a budget below the cheapest conforming basket is reported as infeasible with a reason and a quoted alternative
Product selection and quantities	3, 6, 9, 12	required versus optional categories, minimum and maximum quantities, and optional categories dropped when they do not fit
Modes and scoring	4, 5, 16, 17, 25	Best Value and Lowest Cost differ where they should; missing specifications score neutrally; metrics merge per curated variant; a warranty in days is converted; Best Value cannot lose on the weighted objective it maximises
Cost arithmetic	7, 8, 11, 18, 19, 20	delivery charged once per retailer, discount only above its threshold, monetary precision, and per-line charges that reconcile to the total
Invalid input	10, 27	negative, zero, unparseable amounts, inverted ranges and invalid exchange rates are refused with readable messages
Determinism and alternatives	14, 15, 21, 22	identical input gives the identical basket; alternatives are neither duplicated nor invented; a what-if reports what actually changed
Persistence and currency	23, 24, 26, 28	save and reload, an INR-priced catalogue, refusal instead of conversion when a currency is unsupported, and monotonic conversion
End-to-end UI	BudgetOptimizerUiTest	rail navigation, optimisation, what-if re-optimisation, per-line charges, save and re-open, printable document, infeasible path and invalid input, all inside the running window

Table 7.5: Automated coverage of the budget planner

7.11 Limitations of the Planner

The following limits are displayed in the interface as well as recorded here. Each category receives one product repeated by quantity, because the curated catalogue holds one congruent variant per category rather than several interchangeable models; a shopping list such as a laptop, a mouse, a keyboard and a headset therefore needs four catalogue categories, and the planner offers only the categories that really contain products. Prices are the same historical observations as the comparison screen, so a basket is a plan and not a checkout, and no live price, stock level or delivery date is implied. Delivery fees and discounts are the user's own assumptions, apportioned across lines as estimates rather than retailer invoices, and the rupee figures depend on the rate the user typed because the

project may not fetch or invent an exchange rate. Specification-based scoring depends on what the retailers happened to document, which is why missing criteria are dropped and disclosed instead of being imputed. When the state cap binds, the result is a bounded heuristic and the interface says so. Finally, printing uses the platform print service: choosing a PDF printer there produces a PDF file, but a machine with no print service is told so rather than being given a file that does not exist.

8. CONCLUSION

Tulita demonstrates that a price-comparison application is primarily a data-identity problem rather than a user-interface problem. By treating the curated product variant as the unit of comparison, validating every source row and photograph by hash before display, and refusing to display a card that has fewer than two priced retailers, the system removes the failure mode that makes casual price comparison misleading: showing two different products side by side and calling the cheaper one a better deal.

The object-oriented structure carried the project further than a procedural design would have. A single data-access interface allowed the same controller, service and views to work against local files and MySQL without duplication; immutable records made the validation rules easy to state; an interface over the camera made the hardware path testable without hardware; and a background executor with explicit hand-back to the JavaFX thread kept the interface responsive while the model indexed 237 photographs. Honest failure handling — visible errors instead of fallbacks — turned out to be as valuable during development as it is for the user, because every unsupported state announced itself.

The measured results are modest but real: 79 variants, 161 genuinely priced observations, a median gap of 17.6 % between the lowest and highest recorded amount, and 12 exact ties reported as ties. The automated suite of 133 tests passes, the application runs from an installed image with no JDK present, and image search executes offline inside the JVM.

The budget planner extended the project from comparison to decision support without changing its architecture. Because the optimiser was written as a Pareto-pruned dynamic programme over integer money with an independent constraint validator, the project can make three claims that a sort-by-price implementation could not: a recommendation never exceeds the budget, no mandatory requirement is ever relaxed silently, and every line is explained by the score that produced it. The rupee mode shows the same discipline applied to money: because the catalogue records dollars and the project may not invent an exchange rate, the user supplies the rate and every converted figure is labelled with it, while the search itself remains in the recorded currency so that a rate cannot change the recommendation.

The project also concludes something about honesty in data-driven software. The dataset does not contain ratings or capture dates, so the application says so instead of estimating them, and the chart that would have shown a price trend is replaced by one that shows what was actually recorded. That constraint was more instructive than an extra feature would have been, and it is the part of the design most worth carrying into future work: additional variants per category so that a basket can choose between interchangeable models, a second dataset with dated observations for a genuine trend, a calibrated rejection threshold for image search, and a native Windows installer built on the target operating system.

9. REFERENCES

[1] Macková, K. and Pilát, M. Product-Mapping-Datasets (ProMap): labelled same-product pairs across Amazon, Walmart and Target. GitHub repository, commit 346d62f8ec4d229db6c437d4c2d9101b02e1a8da. https://github.com/kackamac/Product-Mapping-Datasets

[2] Macková, K. and Pilát, M. Product matching on e-commerce platforms. arXiv:2309.06882, 2023. https://arxiv.org/abs/2309.06882

[3] Macková, K. and Pilát, M. Same product, different price: product matching for e-commerce. Electronic Commerce Research, 2024. https://doi.org/10.1007/s10660-024-09892-9

[4] Radford, A. et al. Learning Transferable Visual Models From Natural Language Supervision. Proceedings of ICML 2021; arXiv:2103.00020.

[5] OpenAI. CLIP: Contrastive Language-Image Pre-training, reference implementation. MIT License. https://github.com/openai/CLIP

[6] Xenova. clip-vit-base-patch32, ONNX conversion of the CLIP vision tower (onnx/vision_model_quantized.onnx), commit d15189d7. https://huggingface.co/Xenova/clip-vit-base-patch32

[7] Microsoft. ONNX Runtime 1.20.0, Java API and execution providers. https://onnxruntime.ai/docs/get-started/with-java.html

[8] OpenJFX. JavaFX 21 documentation: controls, charts, concurrency and CSS reference. https://openjfx.io/javadoc/21/

[9] Oracle. Java Platform, Standard Edition 17 API Specification and the jpackage tool specification. https://docs.oracle.com/en/java/javase/21/docs/specs/man/jpackage.html

[10] Oracle. MySQL Connector/J 8.4 Developer Guide and the JDBC 4.2 specification. https://dev.mysql.com/doc/connector-j/en/

[11] Oracle. MySQL 8.0 Reference Manual: InnoDB transactions, isolation levels, check constraints. https://dev.mysql.com/doc/refman/8.0/en/

[12] Sarxos. webcam-capture: Java webcam library. https://github.com/sarxos/webcam-capture

[13] FasterXML. jackson-databind 2.18, data binding for JSON. https://github.com/FasterXML/jackson-databind

[14] The Apache Software Foundation. Apache Commons CSV 1.12.0 User Guide. https://commons.apache.org/proper/commons-csv/

[15] JUnit Team. JUnit 5 User Guide, version 5.11. https://junit.org/junit5/docs/current/user-guide/

[16] Kellerer, H., Pferschy, U. and Pisinger, D. Knapsack Problems. Springer, Berlin, 2004. ISBN 978-3-540-40286-2. (Multiple-choice knapsack formulation and dynamic-programming solution used in Chapter 7.)

[17] Ehrgott, M. Multicriteria Optimization, 2nd edition. Springer, Berlin, 2005. ISBN 978-3-540-21398-7. (Pareto dominance and the pruning rule used by the optimiser.)